



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan f
Bengaluru - 560059, Karnataka,

Go, change the world

Machine Learning Operations (MLOps) Tutorial

Wildfire Risk Prediction System Team No. 6

USN

NAME

1RV23AI086

SANKALP KHAMESRA

1RV23AI085

SAMRUDDHI D

1RV23AI087

SASANK SEKHAR PANDA

1RV23AI089

SHANAVI NARAYAN



Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs



INTRODUCTION

- Wildfires are increasing in frequency and severity due to climate change, prolonged droughts, rising temperatures, and changing land-use patterns, causing major environmental, economic, and human losses.
- Current wildfire management systems are largely reactive, relying on manual monitoring and historical indices, which limits early risk detection and timely preventive action.
- Although large volumes of historical and environmental data are available, existing systems lack intelligent automation and predictive analysis to effectively assess wildfire risk in advance.
- There is a need for a data-driven predictive system that can analyze historical wildfire and environmental data to forecast wildfire risk levels and support proactive decision-making.



MOTIVATION

- The increasing frequency and severity of wildfires pose serious threats to human life, ecosystems, and infrastructure, highlighting the need for proactive and data-driven risk assessment methods.
- Advances in machine learning and the availability of large-scale historical wildfire and environmental datasets create an opportunity to move from reactive wildfire management to predictive and preventive strategies.
- An intelligent wildfire risk prediction system can support early warning, better resource allocation, and informed decision-making, ultimately reducing economic losses and improving public safety.



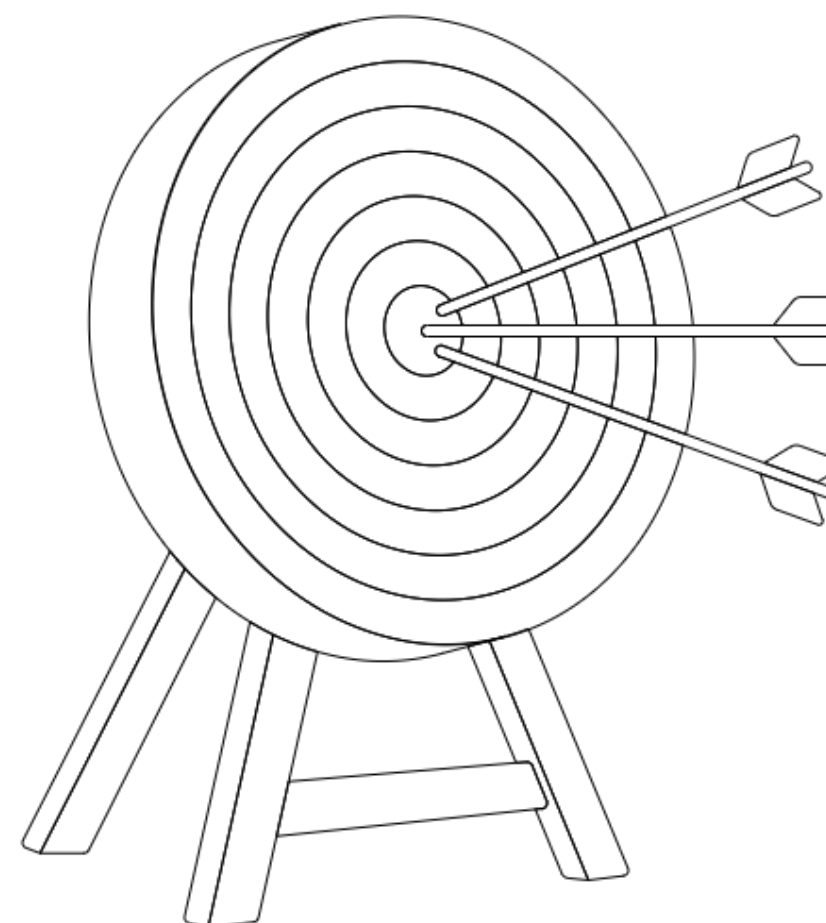
PROBLEM DEFINITION

- Wildfires pose a growing threat to human life, ecosystems, and infrastructure due to increasing climate variability, prolonged droughts, and rising temperatures.
- Existing wildfire management systems primarily rely on reactive and manual monitoring approaches, which are insufficient for anticipating wildfire ignition and risk in advance. Although large volumes of historical wildfire and environmental data are available, they are not effectively utilized to generate timely and reliable risk predictions.
- The lack of an intelligent, data-driven, and predictive framework limits early warning capabilities and informed decision-making.
- Therefore, the problem is to design and develop a scalable machine learning–based system that can analyze historical wildfire and environmental data to accurately predict wildfire risk levels and support proactive wildfire preparedness and mitigation efforts.

OBJECTIVES

- To analyze historical wildfire occurrence data from the US Wildfire Dataset to identify key patterns and trends.
- To develop a predictive model that estimates wildfire risk.
- To evaluate the performance of the proposed model using appropriate accuracy and reliability metrics.

Wildfire Risk Prediction Objectives



Model Evaluation

Assess model performance with metrics

Predictive Model

Develop model to estimate wildfire risk

Data Analysis

Identify patterns and trends in wildfire data

DATASET DESCRIPTION

- The project utilizes the U.S. Spatiotemporal Wildfire Ignition Dataset, developed for large-scale wildfire prediction and spatiotemporal analysis.
- The dataset integrates wildfire ignition events from the Integrated Reporting of Wildland Fire Information (IRWIN) system with daily meteorological and environmental variables obtained from GRIDMET.
- The dataset spans the period 2014 to 2025 and covers the continental United States, providing comprehensive spatial and temporal coverage.
- The dataset contains a total of **126,800 labeled samples**, including **50,720 positive wildfire ignition events** and **76,080 negative samples**.
- When expanded into 75-day sequences, the dataset yields **over 9.5 million daily observation rows**, making it suitable for large-scale sequence learning and spatiotemporal analysis.
- Each observation includes 15 standardized environmental variables capturing atmospheric, fuel, and fire-risk conditions.



US Wildfire Dataset (2014-2025)

9.5 Million Spatiotemporal Wildfire Forecasting Data
(GRIDMET+IRWIN)

 [kaggle.com](https://www.kaggle.com)

RISK ANALYSIS

- The project explicitly addresses data quality, environmental variability, model accuracy, performance, and generalization risks.
- Data versioning and latency risks are considered at a conceptual level through reproducible workflows and batch-based prediction.
- While full automation and real-time deployment are identified as future enhancements.

PROBABILITY	Highly Probable	-	Versioning Risk	Delayed Reporting	Environment Variability
	Probable	Data Risk	Data Completeness	Generalization Risk	Model Drift
	Possible	Label Accuracy Risk	Model Accuracy	Performance Risk	Maintenance Risk
	Unlikely	Data Integration Risk	Cloud Infrastructure Outage	-	Pipeline Failure
		Highly Probable	Probable	Possible	Unlikely
		IMPACT			



TOOLS AND TECHNIQUES USED

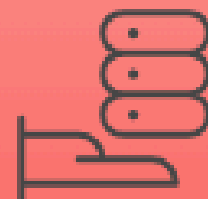
Python

Core programming language for data processing and machine learning.



Uvicorn

Application server for FastAPI backend services.



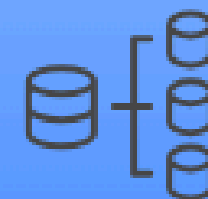
MLflow

Tracks experiments, logs parameters, and manages model versions.



Evidently AI

Monitors data drift by comparing live data with reference datasets.



Docker

Containerizes frontend and backend inference services.



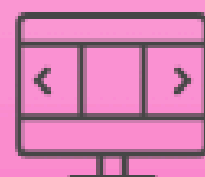
FastAPI

Deploys trained models as RESTful APIs.



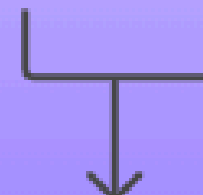
React.js (Vite)

Develops interactive frontend dashboard for visualization.



DVC

Versions datasets and trained models for reproducibility.



Prefect

Orchestrates automated workflows, including retraining pipelines.



Git

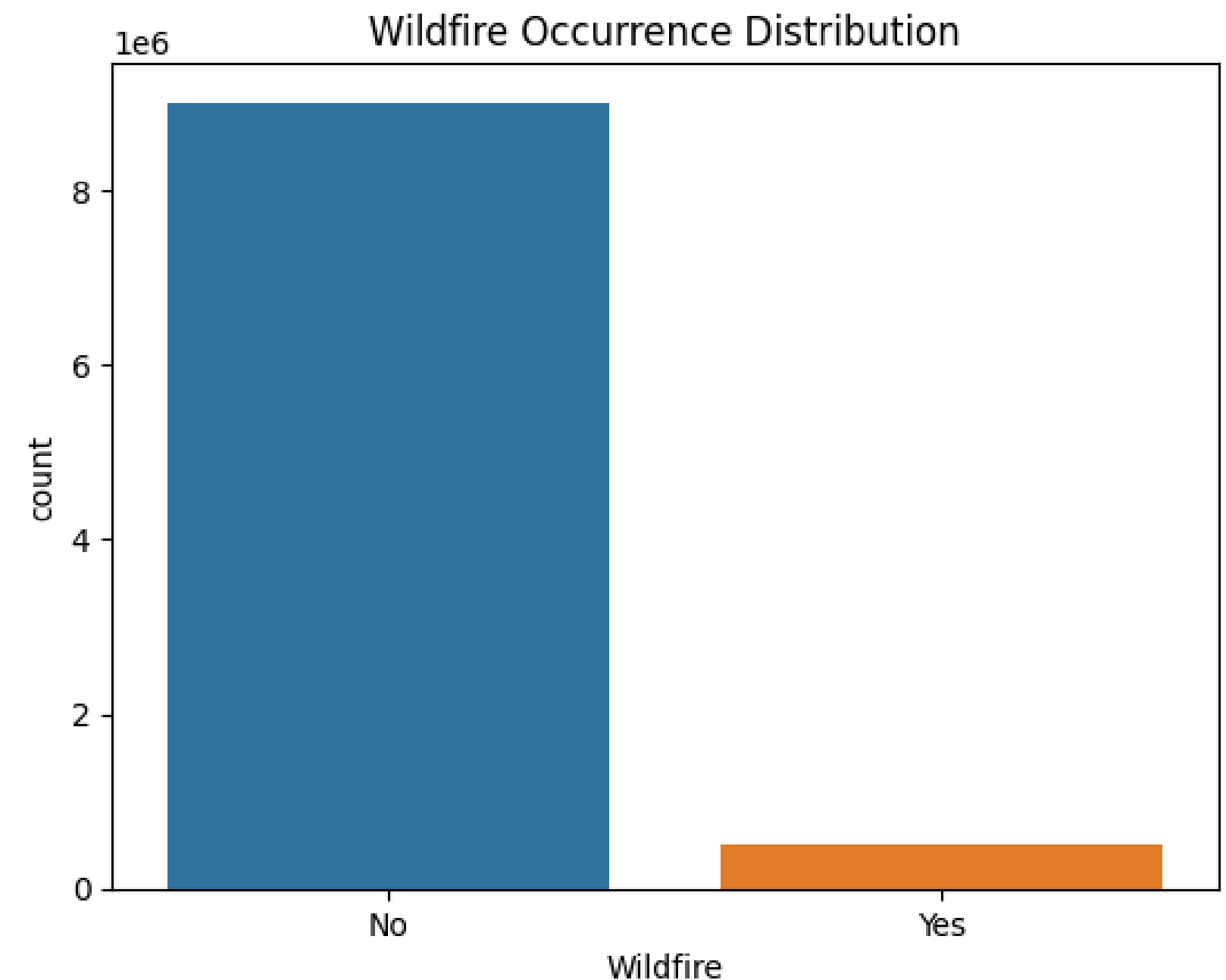
Version control, collaboration, and codebase maintenance.



PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

- Dataset Size: 9,509,925 daily observations with 22 attributes, including latitude, longitude, datetime, meteorological variables, fire indices, and the wildfire label.
- Temporal Coverage: 2014–2025
- Spatial Coverage: Continental United States
- Memory footprint: ~1.5 GB (large-scale spatiotemporal dataset)
- Target Variable: Binary wildfire ignition (Yes/No)
- Class Imbalance: Wildfire events constitute ~5.28% of total observations, with non-wildfire days accounting for ~94.72%. This indicates a strong class imbalance, which is expected in real-world wildfire ignition scenarios.
- Low missing value rates (~0.27%) across features indicate high data quality.
- No missing values in target (Wildfire) or spatiotemporal identifiers



PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

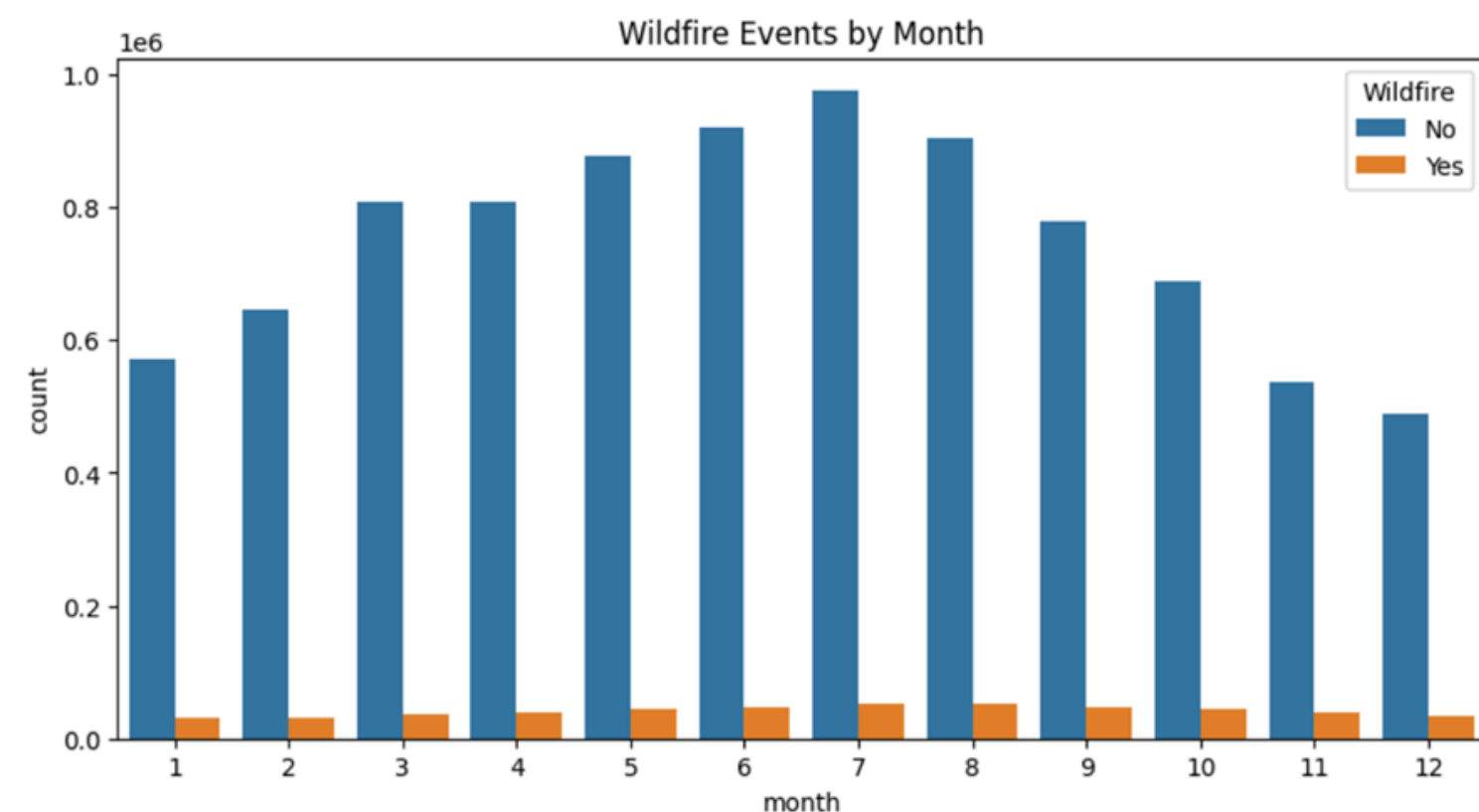
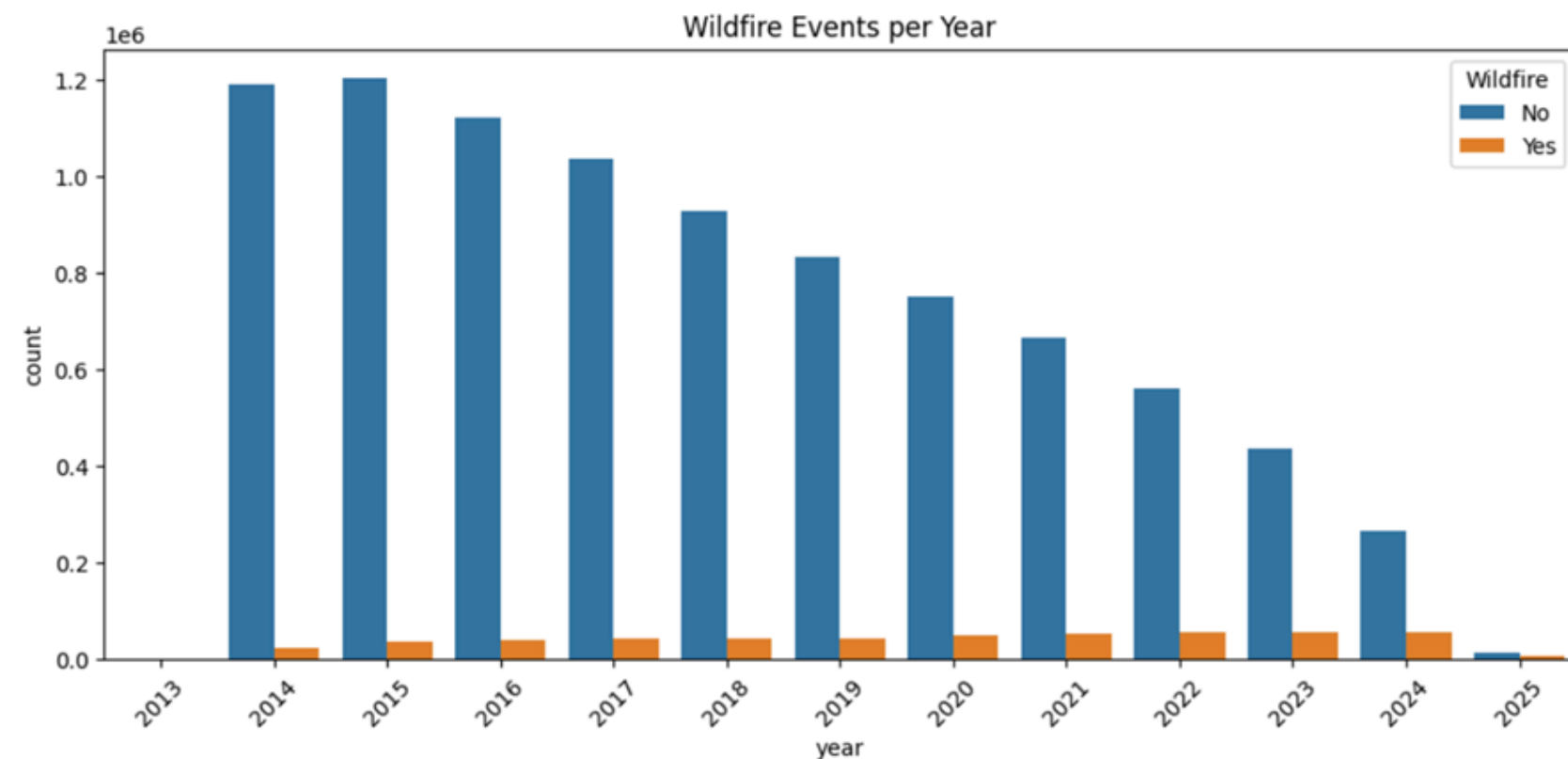
- Weather-related variables such as precipitation, temperature, humidity, wind speed, and fire indices exhibit wide ranges and high standard deviations, indicating diverse climatic and fuel conditions relevant to wildfire risk.
- Several features (e.g., precipitation and evapotranspiration) show skewed distributions with frequent zero values, reflecting dry conditions common in wildfire-prone regions.

	count	mean	std	min	25%	50%	75%	max
latitude	9.510e+06	39.144	5.267	2.526e+01	34.689	38.532	43.670	48.999
longitude	9.510e+06	-106.435	14.569	-1.244e+02	-118.344	-110.926	-95.420	-67.013
pr	9.484e+06	1.737	6.330	0.000e+00	0.000	0.000	0.000	528.000
rmax	9.484e+06	76.144	20.781	5.000e+00	61.700	79.900	95.000	100.000
rmin	9.484e+06	33.524	18.362	1.000e+00	19.300	30.700	45.400	100.000
sph	9.484e+06	0.006	0.003	1.300e-04	0.004	0.006	0.008	0.024
srad	9.484e+06	221.120	88.087	0.000e+00	147.000	227.300	299.600	440.600
tmmn	9.484e+06	280.335	8.749	2.309e+02	274.800	281.100	286.300	310.000
tmmx	9.484e+06	294.301	10.239	2.419e+02	287.800	295.500	301.900	324.300
vs	9.484e+06	3.765	1.678	3.000e-01	2.600	3.400	4.600	19.600
bi	9.484e+06	33.621	23.946	0.000e+00	19.000	34.000	49.000	189.000
fm100	9.484e+06	12.787	4.949	1.100e+00	8.700	12.900	16.500	32.600
fm1000	9.484e+06	14.501	5.217	2.100e+00	10.300	14.800	18.400	42.800
erc	9.484e+06	43.573	24.154	0.000e+00	25.000	39.000	61.000	121.000
etr	9.484e+06	5.634	3.086	0.000e+00	3.200	5.400	7.700	26.500
pet	9.484e+06	4.129	2.236	0.000e+00	2.300	4.000	5.800	16.900
vpd	9.484e+06	1.132	0.882	0.000e+00	0.470	0.920	1.570	8.100

PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

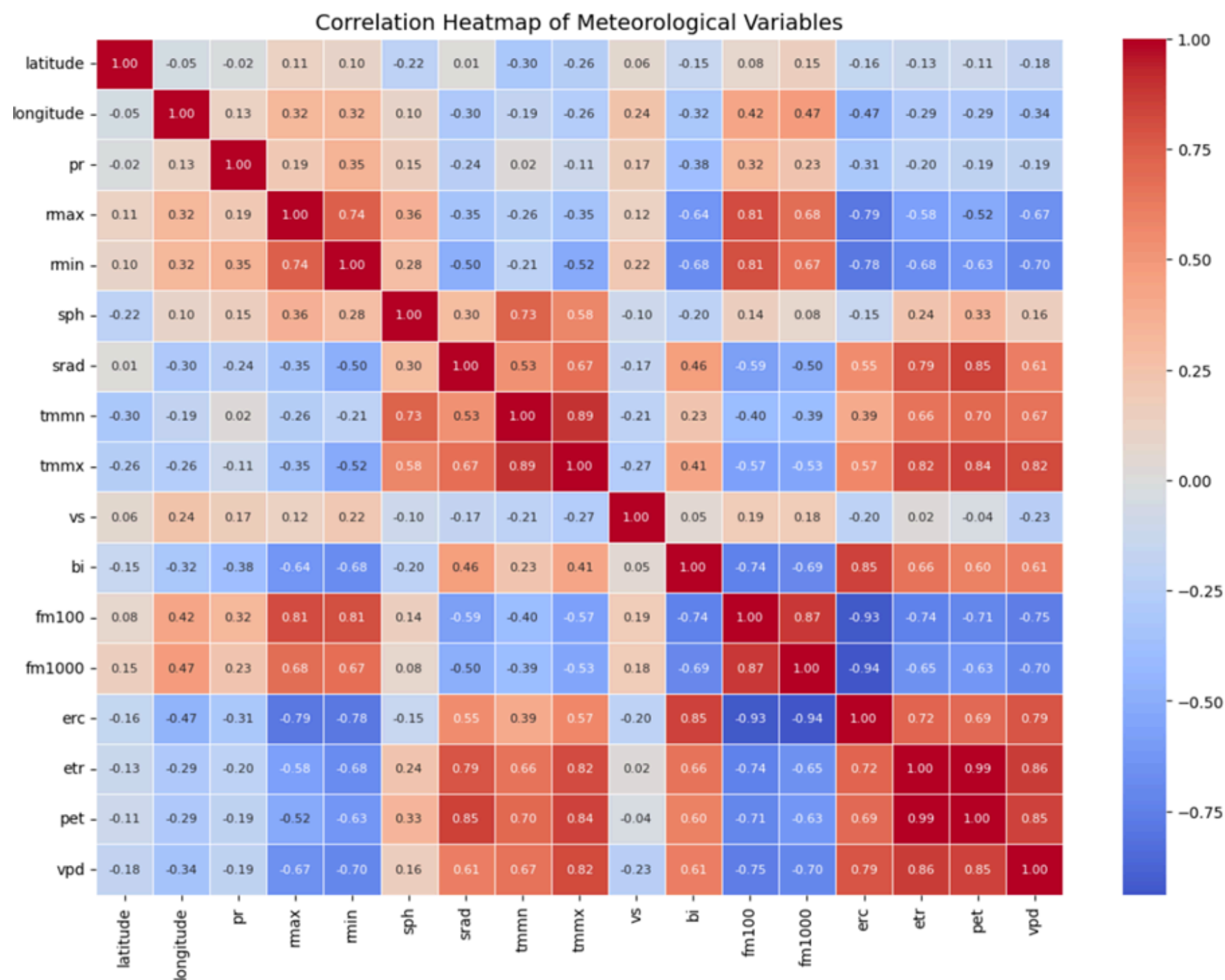
- The year-wise distribution shows that wildfire ignition events occur consistently every year, while non-wildfire observations dominate the dataset. This confirms a strong class imbalance and highlights the necessity of temporal awareness when modeling wildfire risk.
- **Seasonal Patterns:** Month-wise analysis reveals a clear seasonal trend, with wildfire occurrences peaking during the summer months (June–August) and declining during winter, aligning with high temperatures, low humidity, and dry fuel conditions.
- **Temporal Feature Engineering:** Temporal features such as Year, Month, and Day of Year were derived to capture long-term trends, seasonal cycles, and fine-grained temporal variations, enabling the model to learn time-dependent wildfire ignition patterns effectively.



PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

- Strong positive correlations are observed among temperature-related variables (tmmn, tmmx), solar radiation (srad), and evapotranspiration (etr, pet), indicating consistent thermal and energy-driven relationships relevant to wildfire conditions.
- Fire danger indices such as Burning Index (bi) and Energy Release Component (erc) show strong correlations with fuel moisture variables (fm100, fm1000) and temperature, confirming their effectiveness as indicators of wildfire risk.
- Vapor Pressure Deficit (vpd) exhibits strong positive correlation with temperature and evapotranspiration and negative correlation with humidity-related variables, highlighting its importance as a key driver of dry and fire-prone conditions.



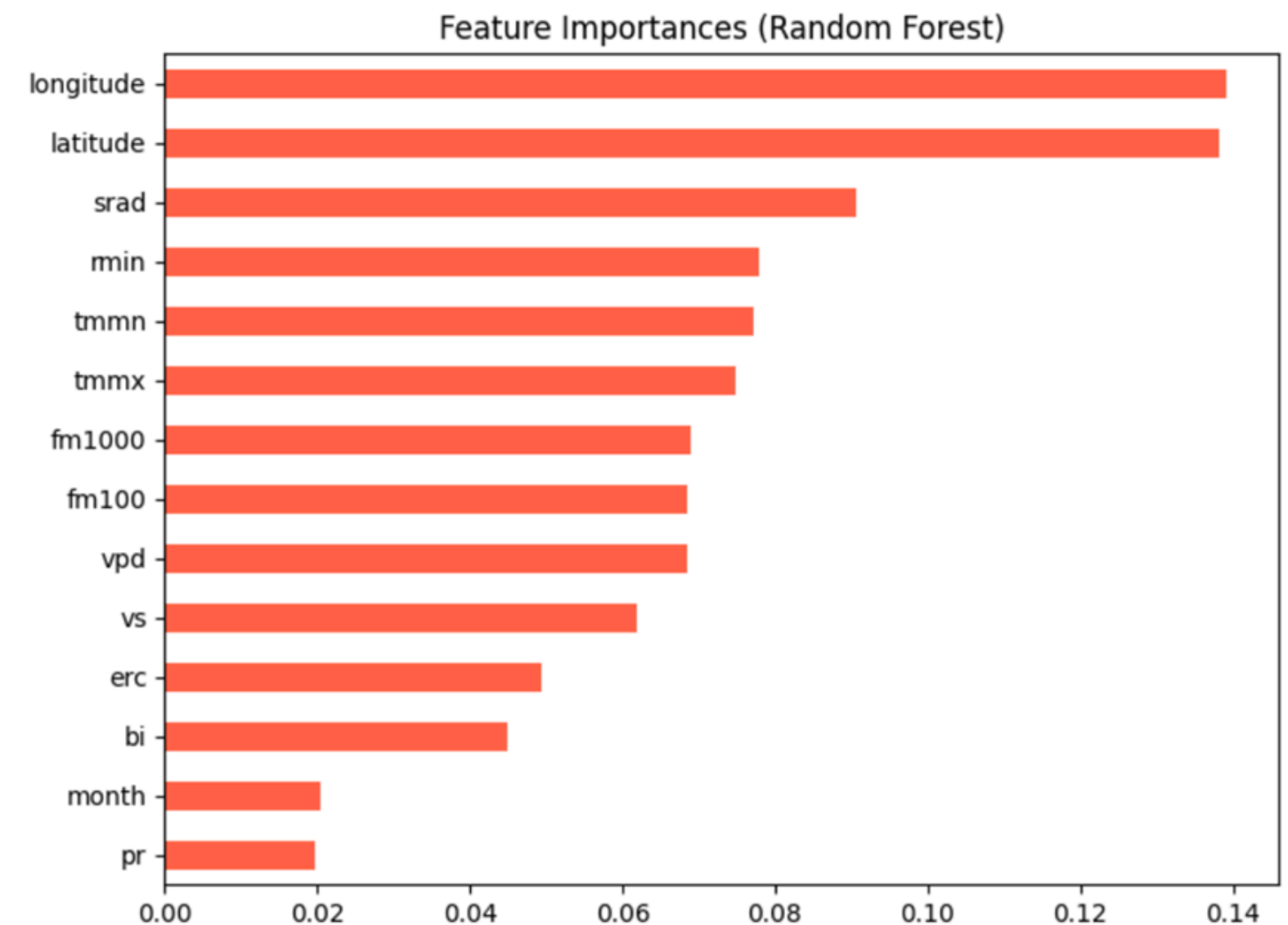
PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

A Random Forest Classifier was trained to evaluate feature importance for wildfire prediction. Top predictors identified:

- Spatial coordinates (latitude, longitude)
- Relative humidity (rmin)
- Temperature (tmmx, tmmn)
- Fuel Moisture (fm100, fm1000)
- Vapor Pressure Deficit (vpd)
- Energy Release Component (ERC)
- Burning Index (BI)

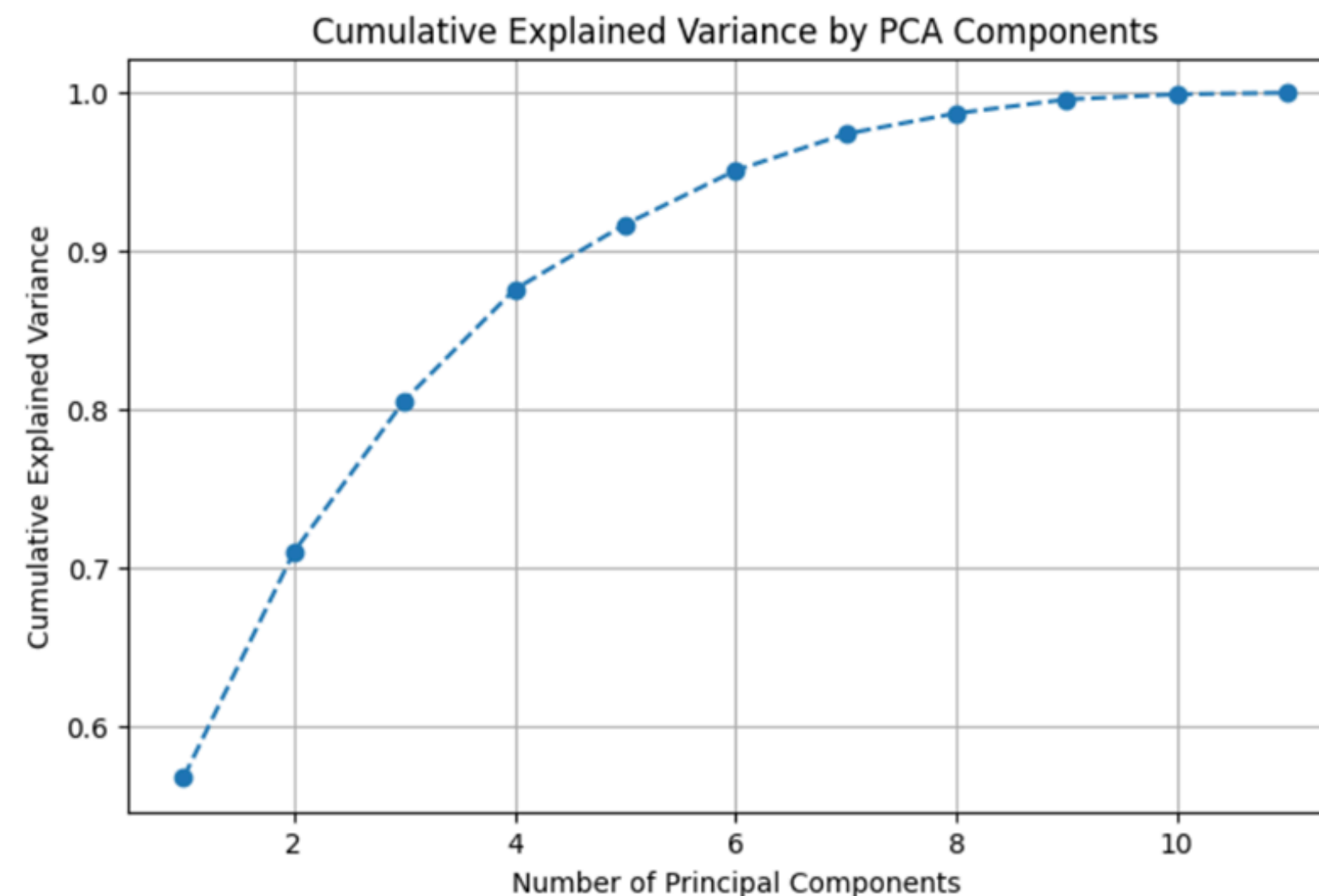
These features contribute most to model decision-making and were prioritized for predictive modeling.



PHASE I - ML LIFE CYCLE

Exploratory Data Analysis (EDA)

- The cumulative explained variance increases rapidly with the first few principal components, indicating that a significant portion of the dataset's variability is captured by a small number of components.
- Approximately 6–7 principal components are sufficient to explain over 95% of the total variance, demonstrating redundancy among the original features.
- PCA helps reduce dimensionality while preserving most of the information, improving computational efficiency and mitigating multicollinearity among highly correlated meteorological variables.



WILDFIRE_RISK

Explore and run machine learning code with Kaggle Notebooks | Using data from US Wildfire Dataset (2014-2025)

 Kaggle / Nov 2, 2025

PHASE I - ML LIFE CYCLE

List of algorithms chosen to build ML model

Random Forest Classifier

- Chosen for its ability to handle non-linear relationships and complex interactions among meteorological, temporal, and geographic features.
- Robust to noise and overfitting due to ensemble learning, making it suitable for large wildfire datasets.

XGBoost (Extreme Gradient Boosting)

- Selected for its high predictive accuracy and efficiency on structured data.
- Effectively handles class imbalance and captures subtle wildfire risk patterns through boosted decision trees.

Long Short-Term Memory (LSTM)

- Chosen to model temporal dependencies in sequential wildfire data.
- Capable of learning long-term trends and seasonal patterns critical for wildfire risk prediction.

Convolutional LSTM (ConvLSTM)

- Selected to capture both spatial and temporal correlations across regions.
- Combines convolutional layers with LSTM to model spatiotemporal wildfire dynamics effectively.

PHASE I - ML LIFE CYCLE

Metrics used to evaluate the ML model

- **Accuracy (Train & Test Accuracy):** Measures the overall correctness of predictions on training and unseen test data, helping assess learning capability and generalization.
- **Recall:** Chosen as a critical metric due to class imbalance, as it measures the model's ability to correctly identify wildfire events (true positives).
- **F1-Score:** Provides a balanced measure of precision and recall, particularly important for evaluating performance on minority wildfire classes.
- **AUC (Area Under the ROC Curve):** Evaluates the model's ability to distinguish between wildfire and non-wildfire classes across different threshold values.
- **Validation Metrics (val_accuracy, val_recall, val_f1, val_auc):** Used to monitor model stability and prevent overfitting during training.
- **System Resource Metrics:** Disk usage and memory utilization (absolute and percentage) were tracked to evaluate the computational efficiency and deployment feasibility of each model.



PHASE I - ML LIFE CYCLE

Evaluation of the ML Models

- The ensemble-based models demonstrate stronger generalization performance compared to deep learning models. XGBoost achieves a test accuracy of approximately 0.69, while Random Forest records a test accuracy of around 0.73. In contrast, LSTM and ConvLSTM models show higher test accuracies ranging between 0.80 and 0.85, indicating challenges in generalization despite higher training accuracy. This improvement is largely driven by the majority non-wildfire class and does not translate to superior performance.
- For wildfire detection, recall is a critical metric due to class imbalance. Random Forest achieves a recall of approximately 0.69, while XGBoost records a recall of about 0.65, demonstrating strong ability to identify wildfire events.
- The AUC scores further confirm this trend, with XGBoost achieving an AUC of 0.67 and Random Forest around 0.65, indicating reliable class separability.
- The F1-scores, though relatively low due to data imbalance, are consistent across models, with XGBoost at approximately 0.14 and Random Forest at 0.13, reflecting the inherent difficulty of minority-class prediction.
- Validation metrics closely align with test metrics. Validation AUC values remain around 0.65–0.67, and validation recall ranges between 0.65 and 0.69, suggesting stable training behavior and minimal overfitting.
- Deep learning models (LSTM and ConvLSTM) consume significantly higher resources, with memory usage exceeding 14 GB ($\approx 85\text{--}88\%$) and disk usage reaching $\sim 7.5\text{--}8$ GB. In comparison, ensemble models demonstrate lower and more stable resource consumption, making them more suitable for scalable and production-ready deployment.



PHASE I - ML LIFE CYCLE

ML Model Chosen for Deployment - RANDOM FOREST

- **Balanced Performance:** Random Forest achieves a strong balance between training and testing performance, with a accuracy of 0.73 and stable generalization, avoiding the overfitting observed in deep learning models such as LSTM and ConvLSTM.
- **High Recall for Wildfire Detection:** The model achieves a recall of approximately 0.69, making it effective at identifying wildfire events, which is critical for early warning and risk mitigation in imbalanced datasets.
- **Robustness to Non-Linear Relationships:** Random Forest effectively captures complex, non-linear interactions among environmental, temporal, and geographic features without requiring extensive feature engineering.
- **Lower Computational and Resource Cost:** Compared to deep learning models, Random Forest requires significantly less memory and disk usage, making it more suitable for scalable and cost-effective deployment.
- **Faster Inference and Simpler Deployment:** The model supports fast inference times and straightforward serialization, enabling smooth integration into REST-based inference pipelines.
- **Interpretability and Feature Importance:** Random Forest provides feature importance scores, improving model transparency and supporting explainable decision-making, which is essential for operational and policy-driven wildfire management systems.



PHASE II - MLOPs LIFE CYCLE

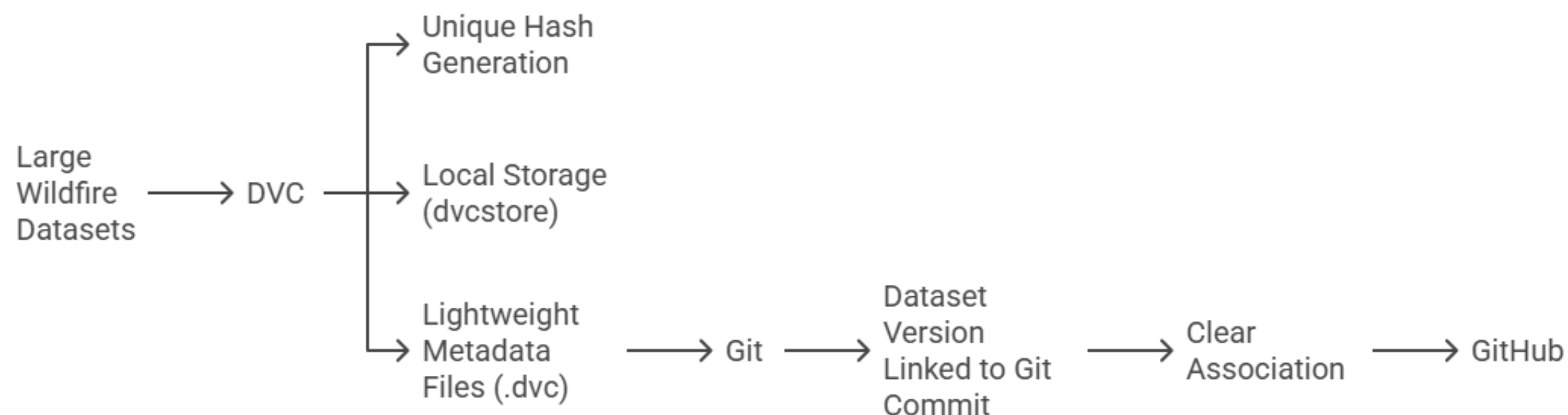
	Component	MLOPs Tools
1	Data Management	DVC , Git
2	Experiment Tracking and Version Control	MLFlow
3	Automation and Pipeline Orchestration	Prefect, GitHub Actions
4	Model Deployment & Serving	Docker, Kubernetes
5	Monitoring & Governance and collaboration	Evidently AI, GitHub, GitHub Actions

PHASE II - MLOPs LIFE CYCLE

1. Data Management (DVC and GIT)

- Data Version Control (DVC) is used to manage and version large wildfire datasets in the project.
- DVC generates a unique hash for each version of the dataset, ensuring data integrity and precise tracking of dataset changes across experiments.
- The actual dataset files are stored locally in a designated DVC storage directory (dvcstore).
- Only lightweight DVC metadata files (.dvc files) are tracked in the repository, keeping the repository efficient.
- Git is used to version control the source code and dataset metadata.
- Each dataset version is linked to a specific Git commit, ensuring reproducibility.
- This setup maintains a clear association between the dataset, preprocessing steps, and model training code.
- The source code and DVC metadata are stored and managed in GitHub, enabling collaborative development and controlled version tracking throughout the project lifecycle.

Data Version Control with DVC



Made with  Napkin



PHASE II - MLOPs LIFE CYCLE

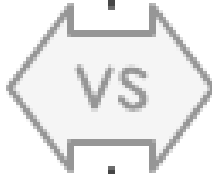
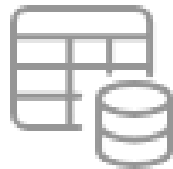
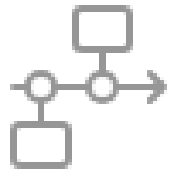
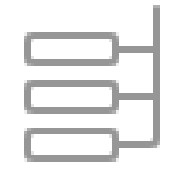
2. Experiment Tracking and Version Control- MLFlow

- MLflow is used to track all machine learning experiments conducted in the project.
- Each model training run is automatically logged as an experiment in MLflow, capturing model parameters, evaluation metrics, and artifacts.
- When a model is trained, it is registered in the MLflow Model Registry, enabling centralized management of model versions.
- The Model Registry maintains multiple versions of the same model, allowing controlled promotion and comparison between models.
- During model retraining triggered by data drift or performance degradation, newly trained models are evaluated and their metrics are compared against existing registered versions.
- Based on performance comparison, the model version in the registry is updated, ensuring that only the best-performing model is selected for deployment.
- This approach provides traceability, reproducibility, and controlled model evolution throughout the project lifecycle.



PHASE II - MLOPs LIFE CYCLE

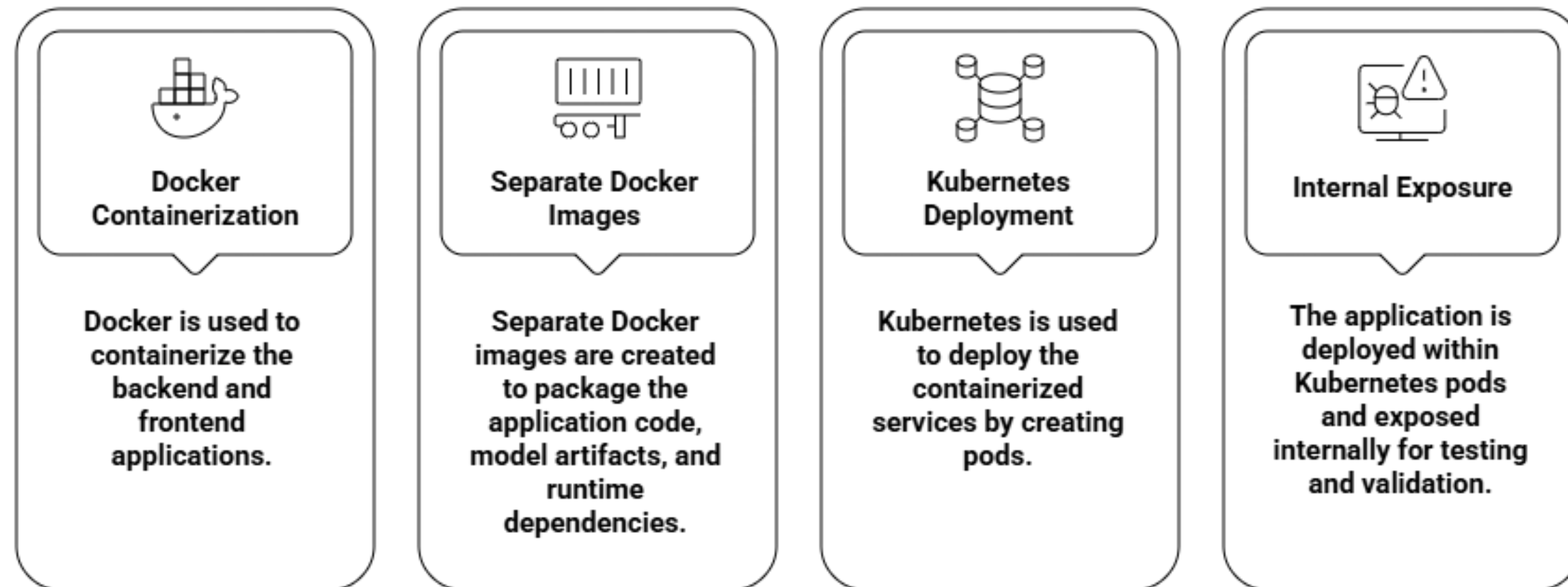
MLFlow Pros And Cons

 Pros	 VS	 Cons
 Centralized tracking		 No orchestration
 Model registry		 Minimal UI
 Metric comparison		 Manual setup
 Python integration		
 Open-source		

PHASE II - MLOPs LIFE CYCLE

3. Model Deployment and Serving

- In this project, Docker is used to containerize both the backend (ML inference service) and the frontend application.
- Separate Docker images are created to package the application code, model artifacts, and all runtime dependencies, ensuring environment consistency.
- Kubernetes is used to deploy these containerized services by creating pods using the local Kubernetes cluster provided by Docker Desktop.
- The application is deployed within Kubernetes pods and exposed internally for testing and validation.

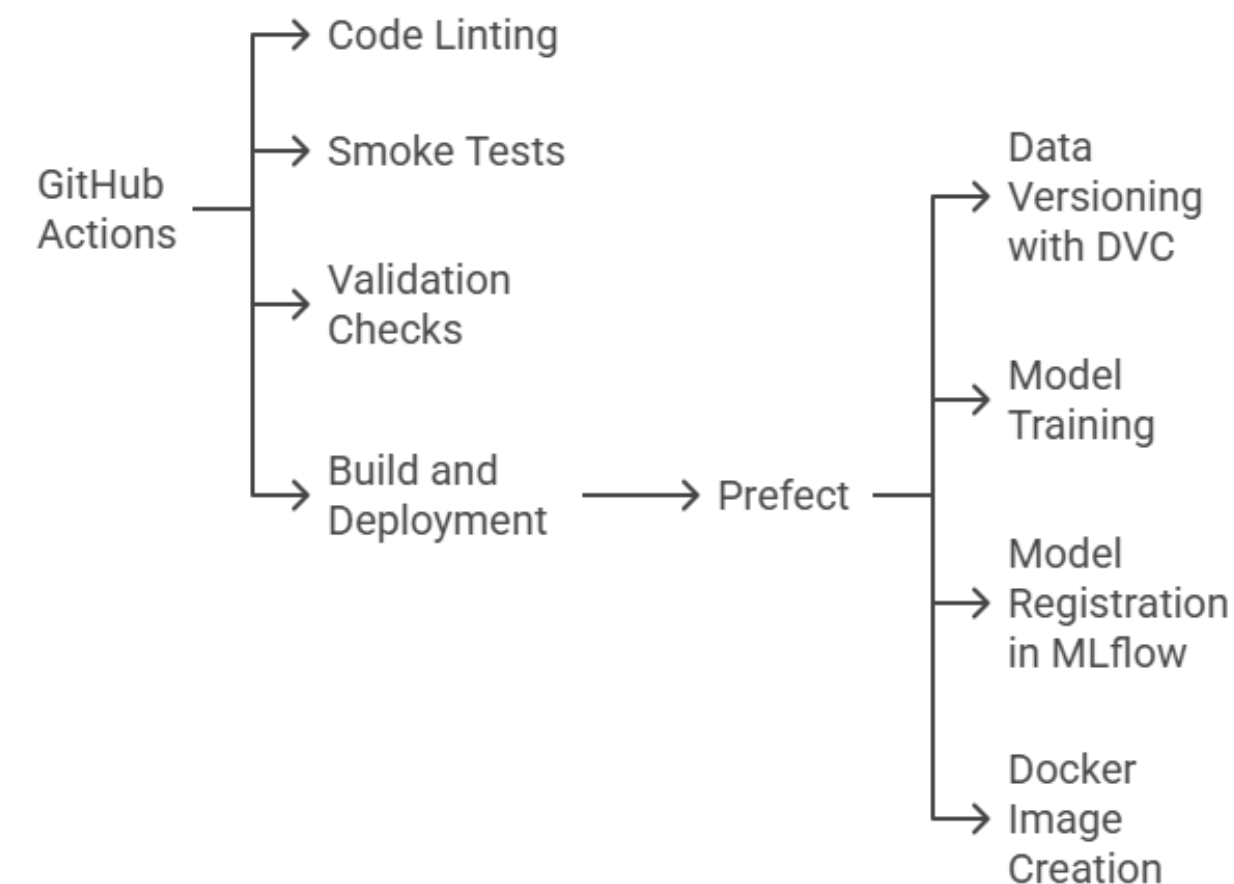


PHASE II - MLOPs LIFE CYCLE

4. Automation and Pipeline Orchestration (Prefect and Github Actions)

- Prefect is used to orchestrate the end-to-end machine learning pipeline in the project.
- The Prefect workflow automates key stages including data versioning with DVC, model training, model registration in MLflow, and Docker image creation for the selected model.
- Each stage of the pipeline is defined as an independent task, enabling modular execution, fault tolerance, and controlled retries.
- Prefect coordinates the complete workflow from data ingestion to model packaging, ensuring consistent and repeatable pipeline execution.
- GitHub Actions is used to automate the CI/CD pipeline, triggering workflows on code commits and pull requests.
- CI workflows include code linting, smoke tests, and validation checks to ensure code quality and pipeline stability.
- GitHub Actions also automates build and deployment steps, integrating seamlessly with the Prefect-orchestrated pipeline.
- This combined setup ensures automation, reproducibility, and continuous integration across the ML lifecycle.

ML Pipeline Orchestration and CI/CD Automation

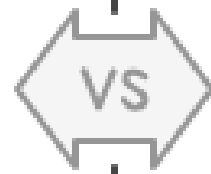
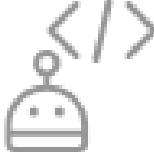


Made with  Napkin



PHASE II - MLOPs LIFE CYCLE

Prefect Pros and Cons

 Pros		 Cons
 Python-native		 Smaller ecosystem
 Robust orchestration		 No native ML tracking
 ML integration		 Not for large scale
 Lightweight		 Immature monitoring
 Local execution		

PHASE II - MLOPs LIFE CYCLE

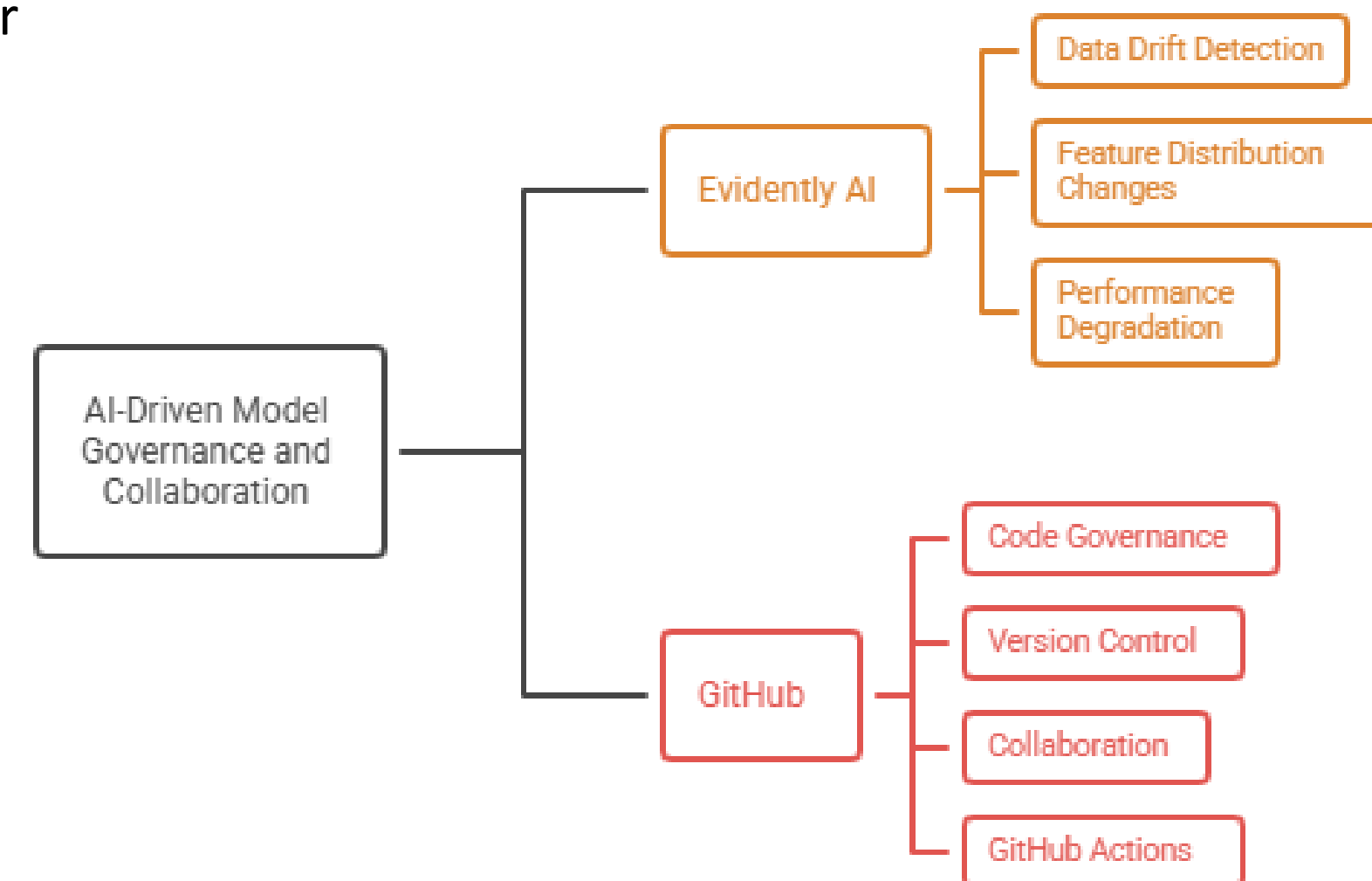
Pros and Cons of Github Actions

Pros	Cons
Native integration with GitHub repositories, unlike Jenkins which requires manual setup.	Limited execution time and compute resources compared to Jenkins and GitLab CI/CD with self-hosted runners.
Minimal setup and no infrastructure maintenance, whereas Jenkins requires continuous server management.	Not suitable for long-running ML training jobs where Jenkins or GitLab CI/CD are better suited.
Simple YAML-based workflows, easier than Jenkins’ Groovy pipelines and comparable to GitLab CI/CD.	Debugging complex workflows is more restricted than in Jenkins.
Large marketplace of reusable actions, reducing custom scripting compared to Jenkins and GitLab CI/CD.	Tightly coupled to GitHub, unlike Jenkins which is platform-agnostic.
Well-suited for CI tasks like linting, testing, and deployment automation without migrating to GitLab.	Less flexibility for highly customized enterprise CI/CD pipelines compared to Jenkins.

PHASE II - MLOPs LIFE CYCLE

5. Monitoring & Governance and collaboration

- Evidently AI is used to monitor the deployed machine learning model for data drift, feature distribution changes, and performance degradation over time.
- Post-deployment data is compared against reference training data using Evidently reports to detect shifts that may impact model reliability.
- Drift detection results are used as a trigger for model retraining, enabling continuous model improvement and governance.
- GitHub is used as the central platform for code governance, version control, and collaboration.
- All changes to code, configuration, and pipelines are tracked through Git commits and pull requests, ensuring transparency and auditability.
- GitHub Actions automates monitoring workflows and validation checks, enabling scheduled or event-based execution of drift detection and reporting.
- This setup ensures model accountability, reproducibility, and collaborative development, supporting responsible ML governance throughout the project lifecycle.



PHASE II - MLOPs LIFE CYCLE

Pros and Cons of EVIDENTLY AI

Pros	Cons
Designed specifically for ML monitoring and drift detection , unlike custom monitoring scripts which require significant development effort.	Does not provide real-time monitoring dashboards like enterprise tools such as Arize AI or Fiddler.
Easy integration with Python ML pipelines, making it simpler to adopt than enterprise-grade monitoring platforms.	Primarily supports batch-based monitoring , whereas managed platforms support streaming data.
Provides clear, interpretable reports for data drift, feature drift, and model performance , unlike generic logging tools.	Limited built-in alerting mechanisms compared to enterprise monitoring solutions.
Open-source and self-hosted, avoiding vendor lock-in unlike cloud-managed monitoring platforms.	Requires manual setup for scheduled execution and report storage.
Lightweight and suitable for academic and proof-of-concept deployments, unlike complex enterprise governance tools.	Not optimized for large-scale, production-grade monitoring pipelines.



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan f
Bengaluru - 560059, Karnataka,

Go, change the world

THANK YOU